

Bags, Vectors & Transformers

Transformers in Practice · Day 3, Part 2

Denise J. Roth, PhD Candidate

Strategic Communication Group · Wageningen University & Research

From Theory to Practice

We know *what* transformers are and *how* attention works. Now the questions that matter for actually **doing research** with them:

- 🏗️ The **pre-train then fine-tune** paradigm — why it changes everything
- 🔧 Three ways to **use** a pre-trained model
- ⚙️ Practical **parameters**: model size, sequence length, variants
- 🌐 Models for **other languages** (including Dutch)
- 👤 **Hugging Face**: how you actually load one in a few lines

Then straight into the notebook.

The Big Idea: Pre-train Once, Fine-tune Often

The reason transformers are usable by *everyone*, not just big labs, is a two-stage split:



The model arrives already knowing **how language works** (grammar, meaning, world knowledge). You just **nudge** it toward your specific task with a modest amount of labeled examples. This is called **transfer learning**.

Why Fine-tuning Is a Big Deal for Social Science

The old way

- Train a model **from scratch**
- Needs **huge** labeled datasets
- Out of reach for most projects
- Lots of compute

With fine-tuning

- Start from a **pre-trained** model
- A **few hundred to a few thousand** labeled examples can suffice
- Runs on a **single GPU** (free in Colab)
- Often beats classic methods

Hand-labeling 500 documents is a realistic afternoon. Hand-labeling a million is not. Fine-tuning meets you where your data is.

Three Ways to Use a Pre-trained Model

1. Off-the-shelf

Use an existing fine-tuned model as-is.

No training. E.g. a ready-made sentiment or topic classifier.

Fastest; only if a model for your task exists.

2. Frozen features

Extract **contextual embeddings**, feed them to a simple classifier.

Model stays **frozen**; you train only the classifier.

Like Day 2, but context-aware vectors.

3. Fine-tuning

Update the **whole model** on your labeled task.

Most powerful, needs a GPU and labels.

Best performance when you have training data.

Increasing effort, increasing power — left to right. Pick the lightest one that solves your problem.

What Can You Use Them For?

Classification tasks

- **Sentiment** & stance detection
- Topic / policy-area coding
- Detecting **incivility**, framing, emotion
- Scaling up hand-coding to a full corpus

Beyond classification

- **Semantic search** & document similarity
- Clustering by meaning
- Named-entity recognition
- Measuring meaning, like Day 2 — but context-aware

The common thread: any task where you would otherwise hand-read thousands of documents.

Practical Parameters to Know

Things that bite you

- **Sequence length:** BERT caps at **512 tokens** — long documents must be split
- **Model size:** more parameters = better but slower
- **GPU:** fine-tuning really wants one
- **Batch size & learning rate:** the main knobs when fine-tuning

Sensible defaults

- Start with a **small model** (DistilBERT) to prototype
- 2–4 fine-tuning **epochs** is usually enough
- Learning rate around **2e-5**
- Always keep a **held-out test set** (Day 1 discipline!)

A Family of Models

“BERT” is really a whole family. You rarely need the original — pick the variant that fits your constraints.

BERT	the original; solid general-purpose baseline
RoBERTa	better-trained BERT; often higher accuracy
DistilBERT	smaller & faster (~40% lighter), minimal accuracy loss — great for prototyping
mBERT / XLM-R	multilingual ; one model, 100+ languages
BERTje / RobBERT	Dutch models, trained on Dutch text

*For Dutch social-science data, **BERTje** or **RobBERT** will usually beat a multilingual model — and both are a click away.*

Hugging Face: The Practical Gateway

You do not train these from scratch or hunt down files. **Hugging Face** is a hub of **thousands of pre-trained models**, plus the transformers library to use them.

The Model Hub

- Search by **task** or **language**
- Ready-made & fine-tuned models
- Model cards document training & limits
- Mostly free to download

The transformers library

- Load any model in **a few lines**
- The pipeline helper for quick tasks
- Full control for fine-tuning
- Works seamlessly in Colab

Sentiment in Three Lines

The pipeline helper hides all the complexity. Here is a working off-the-shelf sentiment classifier:

```
from transformers import pipeline  
clf = pipeline("sentiment-analysis")  
clf("This policy is a disaster.")  
  
# [{'label': 'NEGATIVE', 'score': 0.99}]
```

That is a real, state-of-the-art model running on your text. Swapping "sentiment-analysis" for another task, or naming a specific model (say a Dutch one), is all it takes to change what it does.

Practice: The Takeaways

- ✓ **Pre-train once, fine-tune often:** the model already knows language; you adapt it cheaply
- ✓ Three usage modes: **off-the-shelf, frozen features, fine-tuning** — lightest that works
- ✓ Mind the **512-token limit**, model size, and the need for a GPU when fine-tuning
- ✓ Pick the right **variant** — DistilBERT to prototype, **BERTje/RobBERT** for Dutch
- ✓ **Hugging Face** makes all of this a few lines of code

To the notebook: load a real model from Hugging Face, get contextual vectors, run the “bank” test, and classify text — the payoff of three days.